```
class Functor f => Applicative f where
   pure :: a -> f a        (same as return)
  (<*>) :: f (a -> b) -> f a -> f b          "ap"
( fmap :: (a -> b) -> f a -> f b )

  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
  liftA2 g fa fb = pure g <*> fa <*> fb
    pure g :: f (a -> b -> c)
    pure g <*> fa :: f (b -> c)

  liftA3 g fa fb fc = pure g <*> fa <*> fb <*> fc
  liftA1 :: (a -> b) -> f a -> f b
  liftA1 g fa = pure g <*> fa
```

for a monad m:

```
    pure = return
     (<*>) :: m (a -> b) -> m a -> m b
    mf <*> ma = do
       f <- mf
       a <- ma
       return (f a)
```

Applicative laws:

```
  pure g <*> fa = fmap g fa
   fg <*> pure a = fmap (\g -> g a) fg
 + assoc
```

```
[(+2),(+3)] <*> [0,1]  ->
    [2,3,3,4]
```

```
newtype Ziplist a = Ziplist [a]
 instance Applicative Ziplist where
   Ziplist fs <*> Ziplist as = Ziplist $ zipWith ($) fs as
   pure a = Ziplist $ repeat a
```

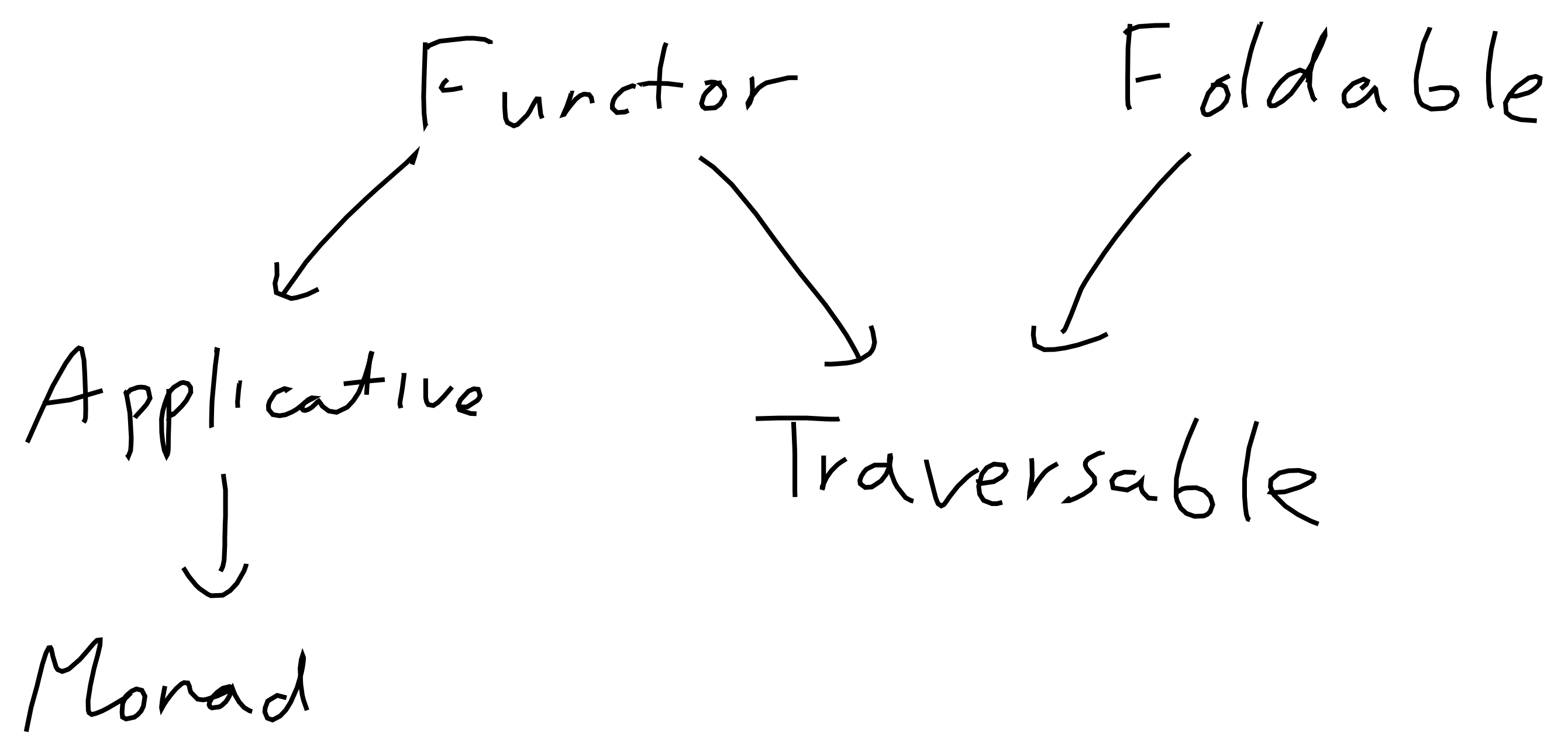
```
  pure g <*> [a_1,a_2] ≠ fmap g [a_1,a_2]
         ||                        ||
  zipWith ($) [g] [a_1,a_2]        ||
              ||                   ||
           [g a_1]     ≠     [g a_1, g a_2]
```

```
newtype Validation e a = Validation (Either e a)

instance Semigroup e => Applicative (Validation e)
                                          where
   pure a = Validation (Right a)
  Val (Right f) <*> Val (Right a) = Val (Right (f a))
  Val (Right f) <*> Val (Left e) = Val (Left e)
  Val (Left e) <*> Val (Right a) = Val (Left e)
  Val (Left e1) <*> Val (Left e2) = Val (Left (e1 <> e2))
```

```
    parseStrs :: [String] -> Validation [Int] [Int]
    parseStrs ss = traverse (\(i,s) -> case parseStrs of
      where                        Nothing -> errVal [i]
        iss :: [(Int, String)]      Just n -> pure n     ) iss
        iss = zip [0..] ss
    parseStr :: String -> Maybe Int
    mapM :: Monad m => (a -> m b) -> [a] -> m [b]
    traverse :: Appl f => (a -> f b) -> [a] -> f [b]
    traverse g [] = pure []
    traverese g (x:xs) = liftA2 (:) (g x) (traverse g xs)
       g x :: f b
      traverse g xs :: f[b]          (:) <$> g x <*> traverse g xs
      (:) :: b -> [b] -> [b]
      liftA2 (:) :: f b -> f[b] -> f[b]

      errVal :: e -> Validation e a
      errVal = Validation . Left
```



```
class (Functor t, Foldable t) => Traversable t
                                          where
   traverse :: Applicative f => (a -> f b) ->
                                   t a -> f (t b)

  data Tree a = Leaf | Node (Tree a) a (Tree a)
 instance Trav Tree where
  traverse g Leaf = pure Leaf
  traverse g (Node l a r) =
       Node <$> traverse g l <*> g a <*> traverse g r

(\a' l' r' -> Node l' a' r') <$> g a <*> traverse g l <*> traverse g r

   cumSum :: Traversable t => t Int -> t Int
   cumSum xs =
       fst $ runState (traverse (\n -> do s <- get
                                          put (s+n)
                                          return (s+n))
                                xs)
                            0
```